Ansible

Configuration management

>> It is the process that ensures every server created in an infrastructure is set up, behaves, works & performs in a uniform way.
>> All servers created will look the same, work & give result in same way
>> It helps maintain consistency, repeatability & reusability across different systems.

Uses of configuration management

1. Ensures consistency
2. Infrastructure scaling
3. Automatic application deployment
4. Automating server configuration

Configuration management tools

1. Ansible
2. Puppet
3. Chef
4. SaltStack

Introduction to ansible

>> It is free and open source automation tool.
>> It is used to configure systems, manage tasks on multiple servers & install software(packages) automatically.

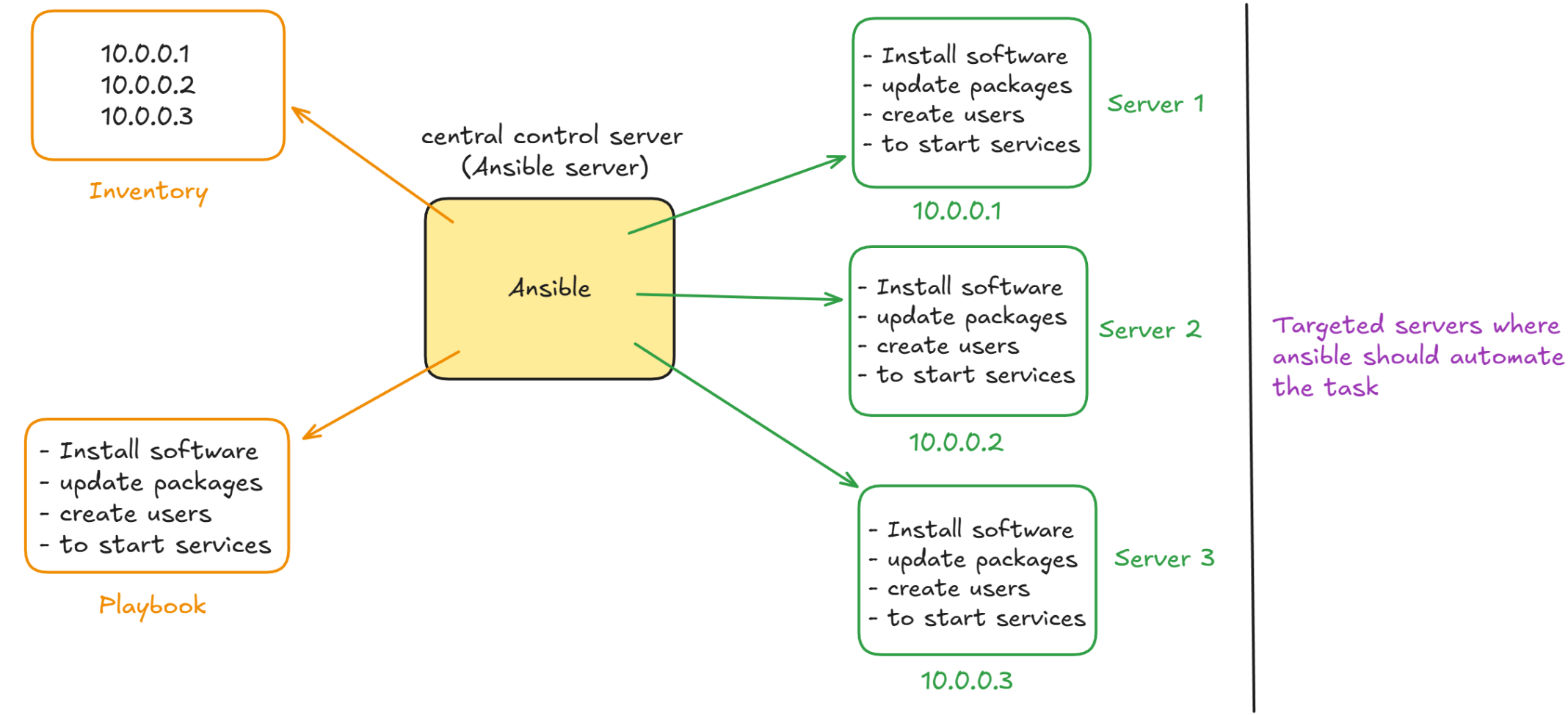
History of ansible

>> Ansible was created by a person named Michael DeHaan in 2012.
>> It was first released as an open source project on GitHub.
>> Due to its growing popularity RedHat acquired ansible.
>> Today ansible is very popular and widely used in DevOps & cloud environment.

Features of ansible

1. It is free and open source
2. Supports multiple platforms (linux, cloud, containers & windows)
3. Secure communication as ansible uses SSH keys
4. Supports scalability
5. Integrates with DevOps tools
6. Easy YAML syntax
7. Uses playbook & inventory files

Architecture of ansible



1. Central control server (Ansible server)

>> It is the main server where the ansible is installed.
>> It is main system from which ansible runs & manages all automation tasks on targeted servers
- It reads the inventory file & executes the playbook files where the instructions are defined
- It will connect to target servers using SSH & will push tasks to target servers.

2. Inventory

>> It is the list of all servers that ansible needs to manage & this file will target the servers to automate the task.
>> It will tell ansible which server to connect to and it contains of IP addresses, DNS name, hostnames or groups.

3. Playbook

>> It is the file where in we write YAML syntax(step by step instructions) that ansible should execute on target servers

4. Target servers

>> Targeted servers are servers where ansible should automate the tasks & applies configuration
- Here ansible shouldn't be installed as it only requires SSH access

Steps to install ansible on AWS EC2 (Ansible Control Server)

1. Go to the AWS console and create a new EC2 instance that will act as the ansible server (Control Node) & While launching the instance, select ubuntu as the operating system, choose an instance type such as t2.micro, t3.micro, t2.medium, or t3.medium, create a new key pair, allow SSH (port 22) in network settings, set storage, launch the instance & connect to the EC2 instance.
2. After connecting update the package repository, install ansible & Verify installation.
   Commands : sudo apt update
   sudo apt install ansible -y
   ansible --version
3. Create a key pair file on the ansible server using the same name as the downloaded key pair.
   Command : nano <keypairname.pem>
4. Paste the content of the downloaded .pem file, save and exit.
5. Change the permissions of the key pair file to allow read-only access for the user.
   Command : chmod 400 <keypairname.pem>

Inventory & Playbook

Inventory

>> Inventory file is used to define the target servers that ansible will connect to from the central control server.

Syntax :

```
[group-name]
# public_IP of instance1
# public_IP of instance2

[group-name:vars]
ansible_user=user_name
ansible_ssh_private_key_file=<path-of-key-pair>
```

- group name :

  It is used to logically group multiple servers & you can run a playbook on all servers in this group at once.

- public_IP of instance1 & instance2 :

  This line represent the public IP address of EC2 instance.

- [group-name:vars] :

  This section is used to define variables for the entire group & any variable defined here applies to all servers inside group name therefore it avoids repeating the same variables for each host.

- ansible_user=user_name :

  It specifies the remote user that ansible will use to connect to the servers.
  For ubuntu EC2 instances the default user is usually ubuntu.

- ansible_ssh_private_key_file=<path-of-key-pair> :

  It specifies the private key (.pem file) used for SSH authentication & ansible uses this key to connect securely to the managed servers.

How to create inventory file & execute it.

1. Create the inventory file
   command : nano inventory
2. Write the script to configure the inventory file
   Ex : [servers]
        43.113.176.65
        84.149.128.14

        [servers:vars]
        ansible_user=ubuntu
        ansible_ssh_private_key_file=/home/ubuntu/keypairname.pem

3. Verify whether ansible is installed correctly and working on the central control server itself.
   command : ansible localhost -m ping
   output : localhost | SUCCESS => {
             "changed": false,
             "ping": "pong"
             }
   - pong means ansible is working fine.
   - to verify ansible installation & to confirm ansible is functional before connecting to remote servers

   Important note : This command does not require inventory file, SSH key, remote servers
   You can use this command before creating the inventory file as well.

4. Pass the command to verify whether the inventory file is correctly targeting and connecting to the servers defined under a specific group & then provide "yes" to confirm verifying the connection.
   command : ansible <group-name> -i inventory -m ping
   output : 43.113.176.65 | SUCCESS => {
            "changed": false,
            "ping": "pong"
            }

            84.149.128.14 | SUCCESS => {
            "changed": false,
            "ping": "pong"
            }

Playbook

>> It is an YAML file that consists of the set of instructions/tasks that needs to be executed in the targeted servers.
>> It is structured, reusable, easy to understand and it automates the tasks.

Syntax :

```
---
- name: <name of playbook>
  hosts: <groupname_for_servers>
  become: yes
  tasks:
      - name: <particular_task_name>
        <module>: <task details>

      - name: <particular_task_name>
        <module>: <task details>
```

1) `---` : Indicates the start of a YAML file & it's recommended best practice for playbooks.

2) `- name: <name of playbook>` : It is the overall name of playbook that helps understand the purpose of playbook while execution.

3) `hosts: <groupname_for_servers>` : It specifies which servers the playbook will run on & the group name must exist in the inventory file.

4) `become: yes` : It enables sudo or root privileges & it's required for installing software or changing permissions etc.

5) `tasks:` : It contains the list of tasks to be executed & here the tasks run sequentially (from top to bottom).

6) `- name: <particular_task_name>` : It describes the specific task being performed.

7) `<module>: <task details>` : It defines the ansible module that performs the action where in task details contain parameters and values for that module.

How to create playbook file to create user.

1. Create the playbook file
   command : nano create_user_playbook.yaml
2. Write an ansible playbook(YAML file) to create a user with a home directory on the targeted servers.
   Playbook file :

```
---
- name: Creating the user with home directory
  hosts: servers
  become: yes
  tasks:
    - name: Create a user
      ansible.builtin.user:
        name: Akshata
        create_home: yes
        shell: /bin/bash
```

3. Execute the ansible playbook using the inventory file to apply the user creation changes on the servers.
   command : ansible-playbook -i inventory create_user_playbook.yaml
4. Connect to target servers and check the users name
   - connect to server
   - pass command to switch to root user --> sudo su
   - pass command to check user --> getent passwd
   - switch to the created user and you can perform any task further.

How to create playbook file to deploy tooplate static website on nginx server .

1. Create the playbook file
   command : nano staticwebsite_playbook.yaml
2. Write an ansible playbook(YAML file) to deploy tooplate static wesite on nginx server.
   Playbook file :

```
---
- name: deploying the static website in nginx server
  hosts: servers
  become: yes
  tasks:
    - name: Install nginx
      ansible.builtin.apt:
        name: nginx
        state: present
        update_cache: yes

    - name: Start service nginx and ensure it is running
      ansible.builtin.service:
        name: nginx
        state: started
        enabled: yes

    - name: Copy file to nginx server
      ansible.builtin.copy:
        src: 2152_event_invitation/
        dest: /var/www/html
        owner: ubuntu
        group: ubuntu
        mode: '765'
```

3. Execute the ansible playbook using the inventory file to apply the user creation changes on the servers.
   command : ansible-playbook -i inventory staticwebsite_playbook.yaml
4. Connect to target servers and check nginx version, nginx status & tooplate folders and files in nginx path.
5. Change the inbound rules of target servers and add rule of http & save it.
6. Copy public_IP of target servers and verify it's accessibility --> http://<public_IP>:80